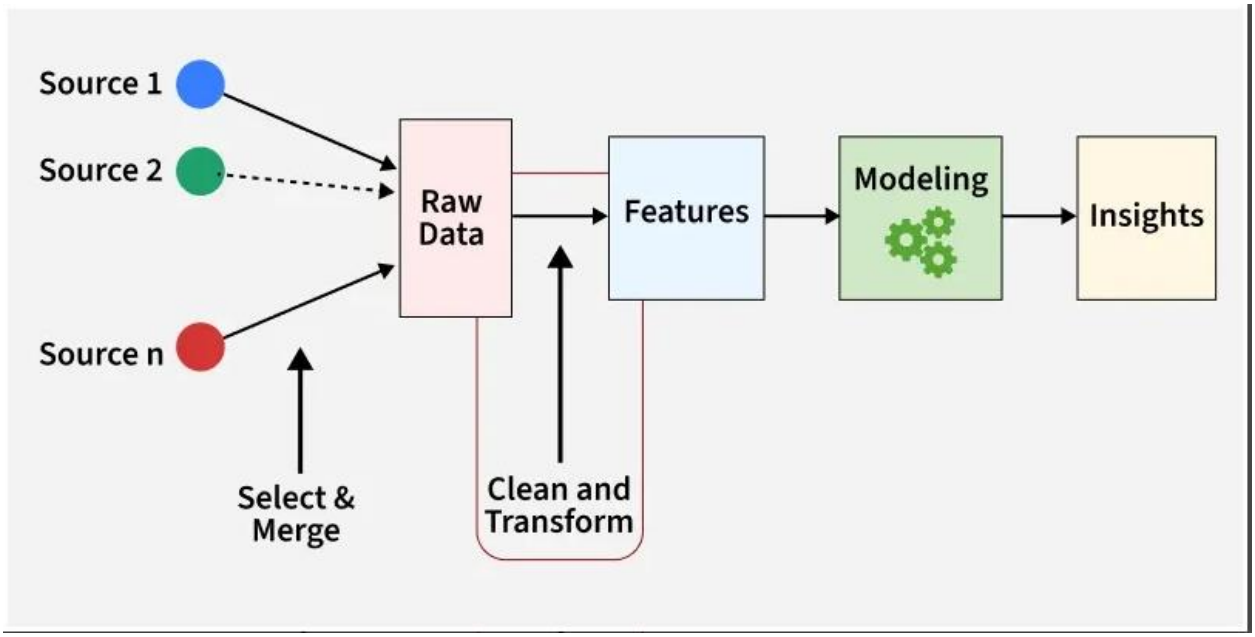
FEATURE ENGINEERING

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively. It involves transforming raw data into meaningful inputs that improve model accuracy and performance.



This step may include handling missing values, encoding categorical variables, scaling numeric variables, creating new features, or combining existing ones. It helps turn messy real-world data into a format that models can understand and use to improve predictions.

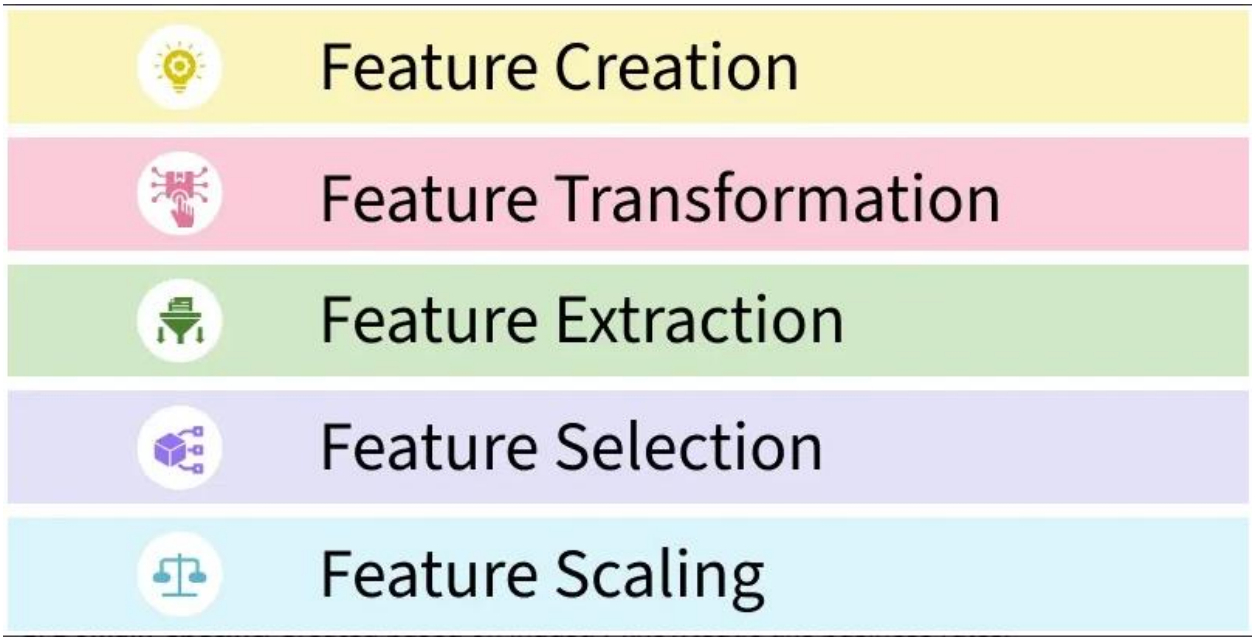
IMPORTANCE OF FEATURE ENGINEERING

Feature engineering can significantly influence model performance. By refining features, we can:

- **Improve accuracy**: Choosing the right features helps the model learn better, leading to more accurate predictions.
- **Reduce overfitting**: Using fewer, more important features helps the model avoid memorizing the data and perform better on new data.
- **Boost interpretability**: Well-chosen features make it easier to understand how the model makes its predictions.
- **Enhance efficiency**: Focusing on key features speeds up the model's training and prediction process, saving time and resources.

PROCESSES INVOLVED IN FEATURE ENGINEERING

Various features involved in feature engineering:



Processes involved in Feature Engineering

1. Feature Creation: Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

1. **Domain-specific**: Created based on industry knowledge, like business rules.
2. **Data-driven**: Derived by recognizing patterns in data.
3. **Synthetic**: Formed by combining existing features.

2. Feature Transformation: Transformation adjusts features to improve model learning:

1. **Normalization & Scaling**: Adjust the range of features for consistency.
2. **Encoding**: Converts categorical data to numerical form, i.e., one-hot encoding.
3. **Mathematical transformations**: Like logarithmic transformations for skewed data.

3. Feature Extraction: Extracting meaningful features can reduce dimensionality and improve model accuracy:

- **Dimensionality reduction**: Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination**: Summing or averaging features to simplify the model.

4. Feature Selection: Feature selection involves choosing a subset of relevant features to use:

- **Filter methods**: Based on statistical measures like correlation.
- **Wrapper methods**: Select based on model performance.
- **Embedded methods**: Feature selection integrated within model training.

5. Feature Scaling: Scaling ensures that all features contribute equally to the model:

- **Min-Max scaling**: Rescales values to a fixed range like 0 to 1.
- **Standard scaling**: Normalizes to have a mean of 0 and variance of 1.

STEPS IN FEATURE ENGINEERING

Feature engineering can vary depending on the specific problem, but the general steps are:

1. Raw data is acquired from various resources and is prepared into a suitable format to be used in the model.
2. **Data Cleaning:** Identify and correct errors or inconsistencies in the dataset to ensure data quality and reliability.
3. **Data Transformation:** Transform raw data into a format suitable for modeling including scaling, normalization and encoding.
4. **Feature Extraction:** Create new features by combining or deriving information from existing ones to provide more meaningful input to the model.
5. **Feature Selection:** Choose the most relevant features for the model using techniques like correlation analysis, mutual information and stepwise regression.

6. **Feature Iteration:** Continuously refine features based on model performance by adding, removing or modifying features for improvement.

FEATURE ENGINEERING TECHNIQUES

One-Hot Encoding:

One Hot Encoding is a ***method for converting categorical variables into a binary format.*** It creates new columns for each category, where **1** means the category is present and **0** means it is not. The primary purpose of One Hot Encoding is to ensure that categorical data can be effectively used in machine learning models.

- Simplest and basic categorical-column encoding method.
- The encoding procedure involves the representation of the element of any finite set by the index of that element in the set, i.e., the element under consideration is assigned to index "1", where all other elements are assigned values within the range of 0 to $n - 1$.
- Now, each of the digits out of $n - 1$ digits is treated as a new column, which allows removal of the original categorical column to which the element belongs.
- It assigns a unique binary number of multiple digits for each possible case or category, making it different from other binary encoding schemes.

Considering we have a feature as color, we need 3 bits, and they are Red, Blue, and Green.
Now, to encode them, we will follow the procedure below.

For every feature value, a fixed position in the array is assigned, where 1 represents a value, and the rest is 0. It is like a vector where the basis is formed by the feature value, and the coefficients form the encoded combination.

Feature	Encoding
Red	100
Blue	010
Green	001

```
import pandas as pd

data = {'Color': ['Red', 'Blue', 'Green', 'Blue']}

df = pd.DataFrame(data)

df_encoded = pd.get_dummies(df, columns=['Color'], prefix='Color')

print(df_encoded)
```

Output

```
   Color_Blue  Color_Green  Color_Red
0       False        False       True
1        True        False      False
2       False         True      False
3        True        False      False
```

2. Binning: Binning transforms continuous variables into discrete bins, making them categorical for easier analysis.

```
import pandas as pd

data = {'Age': [23, 45, 18, 34, 67, 50, 21]}

df = pd.DataFrame(data)

bins = [0, 20, 40, 60, 100]

labels = ['0-20', '21-40', '41-60', '61+']

df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

print(df)
```

Output

```
   Age Age_Group
0   23     21-40
1   45     41-60
2   18      0-20
3   34     21-40
4   67       61+
5   50     41-60
6   21     21-40
```

3. Text Data Preprocessing: Involves removing stop-words, stemming and vectorizing text data to prepare it for machine learning models.

```
import nltk

from nltk.corpus import stopwords

from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer

texts = ["This is a sample sentence.", "Text data preprocessing is important."]

stop_words = set(stopwords.words('english'))

stemmer = PorterStemmer()

vectorizer = CountVectorizer()

def preprocess_text(text):

    words = text.split()

    words = [stemmer.stem(word)

            for word in words if word.lower() not in stop_words]

    return " ".join(words)

cleaned_texts = [preprocess_text(text) for text in texts]

X = vectorizer.fit_transform(cleaned_texts)

print("Cleaned Texts:", cleaned_texts)

print("Vectorized Text:", X.toarray())
```

Output:

```
Cleaned Texts: ['sampl sentence.', 'text data preprocess important.']
Vectorized Text: [[0 0 0 1 1 0]
 [1 1 1 0 0 1]]
```

4. Feature Splitting: Divides a single feature into multiple sub-features, uncovering valuable insights and improving model performance.

```
import pandas as pd

data = {'Full_Address': [

    '123 Elm St, Springfield, 12345', '456 Oak Rd, Shelbyville, 67890']}

df = pd.DataFrame(data)

df[['Street', 'City', 'Zipcode']] = df['Full_Address'].str.extract(r'([0-9]+\s[\w\s]+),\s([\w\s]+),\s(\d+)')
```

print(df)

Output

```
                    Full_Address      Street         City Zipcode
0  123 Elm St, Springfield, 12345  123 Elm St  Springfield   12345
1  456 Oak Rd, Shelbyville, 67890  456 Oak Rd  Shelbyville   67890...
```

TOOLS FOR FEATURE ENGINEERING

There are several tools available for feature engineering. Here are some popular ones:

- **Featuretools**: Automates feature engineering by extracting and transforming features from structured data. It integrates well with libraries like pandas and scikit-learn, making it easy to create complex features without extensive coding.

- **TPOT**: Uses genetic algorithms to optimize machine learning pipelines, automating feature selection and model optimization. It visualizes the entire process, helping you identify the best combination of features and algorithms.

- **DataRobot**: Automates machine learning workflows including feature engineering, model selection and optimization. It supports time-dependent and text data and offers collaborative tools for teams to efficiently work on projects.

- **Alteryx**: Offers a visual interface for building data workflows, simplifying feature extraction, transformation and cleaning. It integrates with popular data sources and its drag-and-drop interface makes it accessible for non-programmers.

- **H2O.ai**: Provides both automated and manual feature engineering tools for a variety of data types. It includes features for scaling, imputation and encoding and offers interactive visualizations to better understand model results.